
PYTHON COMPLETE COURSE

All Commands + Examples + Programs

BY KRUTIK 
CYBER EXPERT

"Learn Free. Code Secure."

CHAPTER 1: PYTHON INTRODUCTION

What is Python?

- High-Level, Interpreted, General-Purpose Programming Language
- Created by Guido van Rossum (1991)
- Current Version: Python 3.12+

Why Python?

- ✓ Easiest to learn
- ✓ Web Development (Django, Flask)
- ✓ AI/ML (TensorFlow, PyTorch)
- ✓ Automation
- ✓ Cyber Security
- ✓ Data Science (Pandas, NumPy)
- ✓ GUI Applications (Tkinter)

CHAPTER 2: INSTALLATION

Step 1: Go to python.org

Step 2: Download Latest Version

Step 3: Install - IMPORTANT: Check "Add Python to PATH"

Step 4: Verify - Open CMD and type: `python --version`

CHAPTER 3: FIRST PROGRAM

CODE:

```
print("Hello World! My name is Krutik 🚓")
```

OUTPUT:

Hello World! My name is Krutik 🚓

CHAPTER 4: VARIABLES

Variable = Container to store data

CODE:

```
name = "Krutik"      # String (Text)
age = 25              # Integer (Number)
price = 99.99         # Float (Decimal)
is_student = True     # Boolean (True/False)
```

```
print(name)
print(age)
print(price)
print(is_student)
```

OUTPUT:

```
Krutik
25
99.99
True
```

Rules for Variable Names:

- Can contain letters, numbers, underscore (_)
- Cannot start with a number
- Case sensitive (name and NAME are different)
- Cannot use reserved keywords (if, else, for, etc.)

CHAPTER 5: DATA TYPES

Data Type	Example	Description
str	"Hello"	Text/String
int	10	Integer Number
float	3.14	Decimal Number
bool	True / False	Boolean
list	[1,2,3]	Collection (mutable)
tuple	(1,2,3)	Collection (immutable)
dict	{"name":"Krutik"}	Key-Value Pairs
set	{1,2,3}	Unique Items
NoneType	None	Empty/Null

Check Data Type using type():

CODE:

```
print(type("Hello")) # <class 'str'>
print(type(10))      # <class 'int'>
print(type(3.14))    # <class 'float'>
print(type(True))    # <class 'bool'>
print(type([1,2,3])) # <class 'list'>
```

CHAPTER 6: TYPE CASTING (Data Type Conversion)

CODE:

String → Integer

```
num = int("123")    # 123
```

Integer → String

```
text = str(456)     # "456"
```

Integer → Float

```
dec = float(10)     # 10.0
```

Float → Integer

```
whole = int(9.99)   # 9 (floor)
```

List → Tuple

```
t = tuple([1,2,3])  # (1,2,3)
```

Tuple → List

```
l = list((1,2,3))   # [1,2,3]
```

Integer → Boolean

```
flag = bool(1)    # True
flag2 = bool(0)   # False
```

CHAPTER 7: USER INPUT

CODE:

```
name = input("Apna naam batao: ")
print("Namaste", name)
```

OUTPUT:

```
Apna naam batao: Krutik
Namaste Krutik
```

IMPORTANT: input() always returns String

To get number, convert:

```
age = int(input("Age batao: "))
price = float(input("Price batao: "))
```

CHAPTER 8: PRINT FUNCTION (Full Details)

CODE:

Basic

```
print("Hello")
```

Multiple Items

```
print("Hello", "World", "Krutik")
```

Separator (default: space)

```
print("Hello", "World", sep="-") # Hello-World
```

```
print("Hello", "World", sep="*") # Hello*World
```

```
# End (default: newline)
```

```
print("Hello", end=" ")
```

```
print("World") # Hello World
```

```
print("Hi", end="")
```

```
print("There") # HiThere
```

```
# Escape Sequences
```

```
print("Hello\nWorld") # \n = new line
```

```
print("Hello\tWorld") # \t = tab
```

```
print("Hello\\World") # \\ = backslash
```

```
print("He said \"Hi\"") # \" = double quote
```

```
print('She said \'Hi\') # \' = single quote
```

CHAPTER 9: STRINGS (All Methods)

CODE:

```
s = "Python Programming"
```

```
# Length
```

```
len(s) # 18
```

```
# Case Conversion
```

```
s.upper() # "PYTHON PROGRAMMING"
```

```
s.lower() # "python programming"
```

```
s.capitalize() # "Python programming"
```

```
s.title() # "Python Programming"
```

```
s.swapcase() # "pYTHON pROGRAMMING"
```

```
# Strip (remove spaces)
```

```
" Hello ".strip() # "Hello"
```

```
" Hello ".lstrip() # "Hello "
```

```
" Hello ".rstrip()    # " Hello"
" Hello ".strip(" ")  # "Hello"
```

Split

```
s.split()              # ['Python', 'Programming']
"a,b,c".split(",")     # ['a','b','c']
"a:b:c".split(":")     # ['a','b','c']
```

Join

```
"-".join(["a","b","c"]) # "a-b-c"
", ".join(["a","b","c"]) # "a, b, c"
```

Replace

```
s.replace("Python", "Java") # "Java Programming"
s.replace("P", "J")         # "Jython Jrogramming"
```

Find and Index

```
s.find("Pro")          # 7 (index)
s.find("XYZ")          # -1 (not found)
s.index("Pro")          # 7 (error if not found)
s.rfind("m")           # 16 (find from right)
```

Count

```
s.count("m")           # 2
s.count("Python")      # 1
```

Startswith / Endswith

```
s.startswith("Py")     # True
s.startswith("Java")   # False
s.endswith("ing")      # True
s.endswith("ing")      # True
```

Check Type

```
"Hello".isalpha()      # True (only letters)
"Hello123".isalpha()   # False
"123".isdigit()        # True
"Hello123".isalnum()   # True (letters + numbers)
```

```
"Hello".isalnum()      # True
"Hello World".isalnum() # False (space)
" ".isspace()          # True
"Hello".islower()      # False
"hello".islower()      # True
"HELLO".isupper()      # True
"Hello".istitle()      # True (first letter capital)
```

Indexing

```
s[0]          # 'P'
s[-1]         # 'g'
s[0:6]        # "Python" (slicing)
s[:6]         # "Python"
s[7:]         # "Programming"
s[7:14]       # "Programm"
s[:2]         # "Pto rgamn" (step 2)
```

Membership

```
"Py" in s      # True
"Java" not in s # True
```

CHAPTER 10: OPERATORS (All Types)

A) ARITHMETIC OPERATORS:

CODE:

```
a = 10
b = 3
```

```
print(a + b) # 13 (Addition)
print(a - b) # 7 (Subtraction)
print(a * b) # 30 (Multiplication)
print(a / b) # 3.333 (Division - Always Float)
```



```
print(a // b) # 3 (Floor Division)
print(a % b) # 1 (Modulus/Remainder)
print(a ** b) # 1000 (Exponent/Power)
```

B) COMPARISON OPERATORS:

CODE:

```
a = 5
```

```
b = 10
```

```
print(a == b) # False (Equal to)
print(a != b) # True (Not equal to)
print(a < b) # True (Less than)
print(a > b) # False (Greater than)
print(a <= b) # True (Less than or equal to)
print(a >= b) # False (Greater than or equal to)
```

C) LOGICAL OPERATORS:

CODE:

```
x = True
```

```
y = False
```

```
print(x and y) # False (Both True)
print(x or y) # True (At least one True)
print(not x) # False (Reverse)
print(not y) # True
```

D) MEMBERSHIP OPERATORS:

CODE:

```
fruits = ["apple", "banana", "mango"]
print("apple" in fruits) # True
print("grape" in fruits) # False
print("grape" not in fruits) # True
```

E) IDENTITY OPERATORS:

CODE:

```
a = [1,2,3]
b = [1,2,3]
c = a
```

```
print(a is b)    # False (different objects)
print(a is c)    # True (same object)
print(a == b)    # True (same value)
print(a is not b) # True
```

F) ASSIGNMENT OPERATORS:

CODE:

```
x = 5
x += 3  # x = x + 3 → 8
x -= 2  # x = x - 2 → 6
x *= 2  # x = x * 2 → 12
x /= 3  # x = x / 3 → 4.0
x //= 2 # x = x // 2 → 2.0
x %= 3  # x = x % 3 → 2.0
x **= 2 # x = x ** 2 → 4.0
```

CHAPTER 11: IF-ELSE (Decision Making)

A) Basic If:

CODE:

```
age = 18
```

```
if age >= 18:
    print("You can vote")
```

B) If-Else:

CODE:

age = 16

if age >= 18:

print("You can vote")

else:

print("You cannot vote")

C) Elif (Multiple Conditions):

CODE:

marks = 85

if marks >= 90:

print("Grade: A+")

elif marks >= 80:

print("Grade: A")

elif marks >= 70:

print("Grade: B")

elif marks >= 60:

print("Grade: C")

else:

print("Grade: F")

D) Nested If:

CODE:

age = 20

has_license = True

if age >= 18:

if has_license:

print("You can drive")

else:

print("Get license first")

```
else:  
    print("Too young to drive")
```

E) Ternary Operator (Short-hand):

```
CODE:  
age = 20  
status = "Adult" if age >= 18 else "Minor"  
print(status) # Adult
```

F) Multiple Conditions:

```
CODE:  
age = 25  
has_id = True  
  
if age >= 18 and has_id:  
    print("You can enter")  
elif age >= 18 and not has_id:  
    print("Need ID")  
else:  
    print("Too young")
```

CHAPTER 12: LOOPS (Repetition)

A) WHILE LOOP:

```
CODE:  
# Basic  
i = 1  
while i <= 5:  
    print(i)  
    i = i + 1 # or i += 1
```

Output: 1 2 3 4 5

Infinite Loop (avoid!)

while True:

print("Hello") # Never ending!

While with break

i = 1

while i <= 10:

if i == 5:

break

print(i)

i += 1

Output: 1 2 3 4

While with continue

i = 0

while i < 5:

i += 1

if i == 3:

continue

print(i)

Output: 1 2 4 5

B) FOR LOOP with range():

CODE:

range(start, stop, step)

for i in range(5): # 0 to 4

print(i)

Output: 0 1 2 3 4

for i in range(2, 8): # 2 to 7

print(i)

Output: 2 3 4 5 6 7

```
for i in range(1, 10, 2): # 1,3,5,7,9
    print(i)
# Output: 1 3 5 7 9
```

```
for i in range(10, 0, -2): # 10,8,6,4,2
    print(i)
# Output: 10 8 6 4 2
```

C) FOR LOOP with List:

CODE:
`fruits = ["apple", "banana", "mango"]`

```
for fruit in fruits:
    print(fruit)
# Output: apple banana mango
```

```
for i, fruit in enumerate(fruits):
    print(i, fruit)
# Output: 0 apple 1 banana 2 mango
```

D) FOR LOOP with String:

CODE:
`name = "Krutik"`

```
for char in name:
    print(char)
# Output: K r u t i k
```

E) FOR LOOP with Dict:

CODE:
`student = {"name": "Krutik", "age": 25}`

```
for key in student:
    print(key, ":", student[key])
```

```
for key, value in student.items():  
    print(key, value)
```

F) FOR LOOP with Tuple and Set:

CODE:

```
# Tuple  
tup = (1,2,3,4,5)  
for num in tup:  
    print(num)
```

```
# Set  
s = {1,2,3,4,5}  
for num in s:  
    print(num)
```

G) BREAK and CONTINUE:

CODE:

```
# Break (loop se bahar)  
for i in range(10):  
    if i == 5:  
        break  
    print(i)  
# Output: 0 1 2 3 4
```

```
# Continue (skip karo)  
for i in range(10):  
    if i % 2 == 0:  
        continue  
    print(i)  
# Output: 1 3 5 7 9
```

H) ELSE with Loop:

CODE:

```
for i in range(5):
    print(i)
else:
    print("Loop finished!")
# Output: 0 1 2 3 4 Loop finished!
```

```
# Else runs only if loop completes normally (no break)
for i in range(5):
    if i == 3:
        break
    print(i)
else:
    print("Loop finished!") # This won't run
# Output: 0 1 2
```

CHAPTER 13: LISTS (All Methods)

A) CREATE LIST:

CODE:

```
fruits = ["apple", "banana", "mango"]
numbers = [1, 2, 3, 4, 5]
mixed = [1, "hello", 3.14, True]
empty = []
range_list = list(range(5)) # [0,1,2,3,4]
string_list = list("Hello") # ['H','e','l','l','o']
```

B) ACCESS ELEMENTS:

CODE:

```
fruits = ["apple", "banana", "mango"]
fruits[0]    # "apple"
fruits[-1]   # "mango"
```



```
fruits[1:3]  # ["banana", "mango"]
fruits[:2]   # ["apple", "banana"]
fruits[1:]   # ["banana", "mango"]
```

C) CHANGE ELEMENTS:

CODE:

```
fruits[1] = "orange"
print(fruits)  # ["apple", "orange", "mango"]
```

D) ADD ELEMENTS:

CODE:

```
# Append - add at end
fruits.append("grape")
print(fruits)  # ["apple", "orange", "mango", "grape"]
```

Insert - add at specific index

```
fruits.insert(1, "kiwi")
print(fruits)  # ["apple", "kiwi", "orange", "mango", "grape"]
```

Extend - add multiple

```
fruits.extend(["peach", "pear"])
print(fruits)  # ["apple", "kiwi", "orange", "mango", "grape",
                 "peach", "pear"]
```

E) REMOVE ELEMENTS:

CODE:

Remove by value

```
fruits.remove("apple")
print(fruits)  # ["kiwi", "orange", "mango", "grape", "peach",
                 "pear"]
```

Pop - remove by index (default last)

```
fruits.pop()          # removes "pear"
fruits.pop(1)          # removes index 1 ("orange")
```

```
print(fruits) # ["kiwi", "mango", "grape", "peach"]
```

```
# Del - delete by index
```

```
del fruits[0] # removes "kiwi"
```

```
print(fruits) # ["mango", "grape", "peach"]
```

```
# Clear - remove all
```

```
fruits.clear()
```

```
print(fruits) # []
```

F) LIST METHODS:

```
CODE:
```

```
fruits = ["apple", "banana", "mango", "apple", "orange"]
```

```
# Length
```

```
len(fruits) # 5
```

```
# Check membership
```

```
"apple" in fruits # True
```

```
"grape" in fruits # False
```

```
# Count
```

```
fruits.count("apple") # 2
```

```
# Index
```

```
fruits.index("mango") # 2
```

```
fruits.index("apple") # 0 (first occurrence)
```

```
# Sort
```

```
fruits.sort() # ascending
```

```
print(fruits) # ["apple", "apple", "banana", "mango", "orange"]
```

```
fruits.sort(reverse=True) # descending
```

```
print(fruits) # ["orange", "mango", "banana", "apple", "apple"]
```

```
# Reverse
```

```
fruits.reverse()
print(fruits) # ["apple", "apple", "banana", "mango", "orange"]
```

```
# Copy
new_fruits = fruits.copy()
print(new_fruits) # ["apple", "apple", "banana", "mango",
"orange"]
```

G) LIST COMPREHENSION:

CODE:

Basic

```
squares = [x**2 for x in range(5)]
print(squares) # [0,1,4,9,16]
```

With condition

```
evens = [x for x in range(10) if x%2==0]
print(evens) # [0,2,4,6,8]
```

Nested

```
matrix = [[j for j in range(3)] for i in range(3)]
print(matrix) # [[0,1,2],[0,1,2],[0,1,2]]
```

H) LIST SLICING (Advanced):

CODE:

```
arr = [0,1,2,3,4,5,6,7,8,9]
```

```
arr[2:7]    # [2,3,4,5,6]
arr[:5]     # [0,1,2,3,4]
arr[5:]     # [5,6,7,8,9]
arr[::2]    # [0,2,4,6,8] (step 2)
arr[::-1]   # [9,8,7,6,5,4,3,2,1,0] (reverse)
arr[1:8:2]  # [1,3,5,7]
```

CHAPTER 14: TUPLES (Immutable Lists)

A) CREATE TUPLE:

CODE:

```
tup = (1, 2, 3, 4, 5)
single = (1,)      # comma important!
empty = ()
tuple_from_list = tuple([1,2,3]) # (1,2,3)
tuple_from_string = tuple("Hello") # ('H','e','l','l','o')
```

B) ACCESS (Same as List):

CODE:

```
tup = (1, 2, 3, 4, 5)
tup[0] # 1
tup[-1] # 5
tup[1:3] # (2,3)
```

C) CANNOT CHANGE (Immutable):

CODE:

```
# tup[0] = 10 ❌ Error! TypeError
```

D) METHODS:

CODE:

```
tup = (1, 2, 2, 3, 4, 2)
tup.count(2) # 3 (count of 2)
tup.index(3) # 3 (index of 3)
len(tup) # 6
```

E) TUPLE UNPACKING:

CODE:

```
a, b, c = (1, 2, 3)
print(a, b, c)    # 1 2 3
```

With star (remaining items)

```
a, *b = (1, 2, 3, 4)
print(a)  # 1
print(b)  # [2, 3, 4]
```

F) TUPLE VS LIST:

CODE:

```
# Tuple is faster and uses less memory
# Use tuple when data shouldn't change
```

CHAPTER 15: SETS (Unique Items)

A) CREATE SET:

CODE:

```
s = {1, 2, 3, 4, 5}
s2 = set([1, 2, 3, 3])  # {1, 2, 3} (duplicate removed)
empty = set()          # {} creates empty dict, not set!
```

B) ADD AND REMOVE:

CODE:

```
s = {1, 2, 3}

# Add
s.add(4)
s.add(4)  # No effect (already exists)
print(s)  # {1, 2, 3, 4}
```

```
# Add multiple
s.update([5, 6, 7])
print(s)  # {1,2,3,4,5,6,7}
```

```
# Remove
s.remove(3)  # Error if not present
s.discard(10) # No error if not present
s.pop()      # Remove and return random element
s.clear()    # Remove all
```

C) SET OPERATIONS:

CODE:

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
```

```
# Union (all elements)
A.union(B)          # {1,2,3,4,5,6}
A | B               # {1,2,3,4,5,6}
```

```
# Intersection (common elements)
A.intersection(B)   # {3,4}
A & B               # {3,4}
```

```
# Difference (in A but not in B)
A.difference(B)     # {1,2}
A - B               # {1,2}
```

```
# Symmetric Difference (in either but not both)
A.symmetric_difference(B) # {1,2,5,6}
A ^ B               # {1,2,5,6}
```

```
# Subset / Superset
A.issubset(B)       # False
A.issuperset(B)     # False
A.issuperset({1,2}) # True
```

D) MEMBERSHIP:

CODE:

```
A = {1,2,3,4}
```

```
1 in A    # True
```

```
5 in A    # False
```

CHAPTER 16: DICTIONARIES (Key-Value Pairs)

A) CREATE DICTIONARY:

CODE:

```
student = {  
    "name": "Krutik",  
    "age": 25,  
    "course": "Python",  
    "grades": [90, 85, 88],  
    "address": {  
        "city": "Mumbai",  
        "pin": 400001  
    }  
}
```

```
empty = {}
```

B) ACCESS ELEMENTS:

CODE:

```
student["name"]          # "Krutik"  
student.get("name")      # "Krutik"  
student.get("city", "N/A") # "N/A" (default)  
student.get("city", "Mumbai") # "Mumbai"  
student["grades"][1]     # 85
```

```
student["address"]["city"] # "Mumbai"
```

C) CHANGE / ADD:

CODE:

```
student["age"] = 26      # Change  
student["city"] = "Mumbai" # Add new  
student.update({"phone": 1234567890, "age": 27}) # Multiple
```

D) CHECK EXISTENCE:

CODE:

```
"name" in student      # True  
"city" not in student  # False
```

E) REMOVE:

CODE:

```
del student["city"]  
student.pop("age")      # Remove and return value (25)  
student.popitem()       # Remove and return last item
```

F) METHODS:

CODE:

```
student = {"name": "Krutik", "age": 25, "course": "Python"}  
  
student.keys()      # dict_keys(['name', 'age', 'course'])  
student.values()    # dict_values(['Krutik', 25, 'Python'])  
student.items()     # dict_items([('name', 'Krutik'), ('age', 25),  
('course', 'Python')])
```

G) LOOP THROUGH DICTIONARY:

CODE:

```
for key in student:  
    print(key, student[key])
```



```
for key, value in student.items():  
    print(key, value)
```

H) DICTIONARY COMPREHENSION:

CODE:

```
squares = {x: x**2 for x in range(5)}  
print(squares) # {0:0, 1:1, 2:4, 3:9, 4:16}
```

Filter

```
even_squares = {x: x**2 for x in range(10) if x%2==0}  
print(even_squares) # {0:0, 2:4, 4:16, 6:36, 8:64}
```

CHAPTER 17: FUNCTIONS

A) BASIC FUNCTION:

CODE:

```
# Define  
def greet():  
    print("Hello! Namaste!")
```

Call

```
greet() # Hello! Namaste!
```

B) WITH PARAMETERS:

CODE:

```
def greet(name):  
    print(f"Hello {name}!")
```

```
greet("Krutik") # Hello Krutik!
```

```
greet("Rahul") # Hello Rahul!
```

C) WITH RETURN VALUE:

CODE:

```
def add(a, b):  
    return a + b
```

```
result = add(5, 10)  
print(result) # 15
```

```
def multiply(a, b):  
    return a * b
```

```
print(multiply(4, 5)) # 20
```

D) DEFAULT PARAMETERS:

CODE:

```
def greet(name="Guest"):  
    print(f"Hello {name}!")
```

```
greet() # Hello Guest!  
greet("Krutik") # Hello Krutik!
```

```
def power(base, exponent=2):  
    return base ** exponent
```

```
print(power(5)) # 25  
print(power(5, 3)) # 125
```

E) MULTIPLE PARAMETERS (*args):

CODE:

```
def add_all(*args):  
    return sum(args)
```

```
print(add_all(1,2,3,4,5)) # 15
print(add_all(10,20))    # 30
```

```
def print_all(*args):
    for item in args:
        print(item)
```

```
print_all("a", "b", "c", "d") # a b c d
```

F) KEYWORD ARGUMENTS (kwargs):**

CODE:

```
def print_info(**kwargs):
    for key, value in kwargs.items():
        print(key, ":", value)
```

```
print_info(name="Krutik", age=25, city="Mumbai")
# Output: name : Krutik, age : 25, city : Mumbai
```

G) LAMBDA FUNCTIONS:

CODE:

```
# Simple lambda
square = lambda x: x**2
print(square(5)) # 25
```

```
# Lambda with multiple parameters
add = lambda a, b: a + b
print(add(10, 20)) # 30
```

```
# Lambda with filter
nums = [1,2,3,4,5,6]
evens = list(filter(lambda x: x%2==0, nums))
print(evens) # [2,4,6]
```

```
# Lambda with map
doubled = list(map(lambda x: x*2, nums))
```

```
print(doubled) # [2,4,6,8,10,12]
```

```
# Lambda with sorted
```

```
students = [("Krutik", 25), ("Rahul", 22), ("Amit", 28)]
```

```
sorted_by_age = sorted(students, key=lambda x: x[1])
```

```
print(sorted_by_age) # [('Rahul',22), ('Krutik',25), ('Amit',28)]
```

H) GLOBAL vs LOCAL VARIABLES:

CODE:

```
x = 10 # Global
```

```
def my_func():
```

```
    x = 5 # Local
```

```
    print(x)
```

```
my_func() # 5
```

```
print(x) # 10
```

```
# Using global keyword
```

```
def change_global():
```

```
    global x
```

```
    x = 20
```

```
change_global()
```

```
print(x) # 20
```

I) NESTED FUNCTIONS & CLOSURES:

CODE:

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        print(x)
```

```
    return inner
```

```
func = outer()
func() # 10
```

J) RECURSION:

CODE:

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n-1)
```

```
print(factorial(5)) # 120
```

CHAPTER 18: FILE HANDLING

A) WRITE TO FILE:

CODE:

```
# Write (overwrite)
f = open("test.txt", "w")
f.write("Hello World!\n")
f.write("Krutik Cyber Expert\n")
f.close()
```

```
# Append (add at end)
f = open("test.txt", "a")
f.write("This is appended\n")
f.close()
```

```
# With statement (auto close - Recommended)
with open("test.txt", "w") as f:
    f.write("Auto closed!\n")
```

```
f.write("No need to call close()\n")
```

Write multiple lines

```
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]
```

```
with open("test.txt", "w") as f:
```

```
    f.writelines(lines)
```

B) READ FROM FILE:

CODE:

Read entire file

```
with open("test.txt", "r") as f:
```

```
    content = f.read()
```

```
    print(content)
```

Read line by line

```
with open("test.txt", "r") as f:
```

```
    for line in f:
```

```
        print(line.strip())
```

Read all lines in list

```
with open("test.txt", "r") as f:
```

```
    lines = f.readlines()
```

```
    print(lines)
```

Read one line at a time

```
with open("test.txt", "r") as f:
```

```
    line1 = f.readline()
```

```
    line2 = f.readline()
```

```
    print(line1, line2)
```

C) FILE MODES:

Mode	Description
------	-------------

"r"	Read (default) - Error if file doesn't exist	
"w"	Write - Creates/Overwrites file	
"a"	Append - Adds at end	
"x"	Exclusive create - Error if exists	
"b"	Binary mode (rb, wb, ab)	
"t"	Text mode (default)	
"+"	Read and Write (r+, w+, a+)	

D) BINARY FILES:

CODE:

Write binary

data = b"Hello Binary"

with open("test.bin", "wb") as f:

f.write(data)

Read binary

with open("test.bin", "rb") as f:

data = f.read()

print(data)

E) FILE OPERATIONS (os module):

CODE:

import os

Check if file exists

os.path.exists("test.txt") # True/False

Get file size

os.path.getsize("test.txt")

```
# Rename
os.rename("test.txt", "new.txt")
```

```
# Delete
os.remove("new.txt")
```

```
# Create directory
os.mkdir("my_folder")
```

```
# List files
os.listdir(".")
```

CHAPTER 19: EXCEPTION HANDLING (Error Handling)

A) TRY-EXCEPT:

CODE:

```
try:
    num = int(input("Enter a number: "))
    print(10 / num)
except ValueError:
    print("That's not a valid number!")
except ZeroDivisionError:
    print("Can't divide by zero!")
except Exception as e:
    print(f"Some error: {e}")
else:
    print("No error occurred!") # Runs if no exception
finally:
    print("This always runs!") # Always runs
```

B) SPECIFIC EXCEPTIONS:

CODE:

try:

x = 10 / 0

except ZeroDivisionError:

print("Divide by zero!")

try:

my_list = [1,2,3]

print(my_list[5])

except IndexError:

print("Index out of range!")

try:

my_dict = {"name": "Krutik"}

print(my_dict["age"])

except KeyError:

print("Key not found!")

try:

int("abc")

except ValueError:

print("Invalid value!")

C) RAISE EXCEPTION:

CODE:

def check_age(age):

if age < 0:

raise ValueError("Age cannot be negative!")

if age < 18:

raise Exception("You are too young!")

return "You are eligible"

try:

print(check_age(-5))

except ValueError as e:

print(f"Error: {e}")

```
try:
    print(check_age(16))
except Exception as e:
    print(f"Error: {e}")
```

D) CUSTOM EXCEPTION:

CODE:

```
class AgeTooYoungError(Exception):
    pass

def check_age(age):
    if age < 18:
        raise AgeTooYoungError("Age must be 18+")
    return "OK"
```

```
try:
    check_age(16)
except AgeTooYoungError as e:
    print(e)
```

CHAPTER 20: MODULES AND PACKAGES

A) IMPORT MODULES:

CODE:

```
# Import entire module
import math
print(math.sqrt(16))    # 4.0
print(math.pi)         # 3.14159
print(math.floor(3.9))  # 3
print(math.ceil(3.1))   # 4
```

```
# Import specific functions
from math import sqrt, pi, factorial
print(sqrt(25)) # 5.0
print(pi) # 3.14159
print(factorial(5)) # 120
```

```
# Rename
import math as m
print(m.sqrt(9)) # 3.0
```

```
# Import all (NOT recommended)
from math import *
print(sin(0)) # 0.0
```

B) POPULAR BUILT-IN MODULES:

CODE:

```
import random
import datetime
import os
import sys
import json
import re
```

```
# Random
print(random.randint(1, 10)) # Random number 1-10
print(random.choice(["a","b","c"])) # Random choice
print(random.shuffle([1,2,3,4,5])) # Shuffle list
print(random.random()) # Random float 0-1
```

```
# Datetime
now = datetime.datetime.now()
print(now.year, now.month, now.day)
print(now.strftime("%Y-%m-%d %H:%M:%S"))
```

```
# OS
```

```
print(os.getcwd())      # Current directory
print(os.listdir("."))  # List files
print(os.path.exists(".")) # Check if exists

# Sys
print(sys.version)      # Python version
print(sys.platform)     # OS platform

# JSON
data = {"name": "Krutik", "age": 25}
json_str = json.dumps(data) # Convert to JSON string
json_obj = json.loads(json_str) # Convert back

# Regular Expressions (re)
import re
text = "My phone is 9876543210"
match = re.search(r'\d{10}', text)
print(match.group()) # 9876543210
```

C) CREATE YOUR OWN MODULE:

Save as "mymodule.py":

CODE:

```
def greet(name):
    return f"Hello {name}!"
```

```
def add(a, b):
    return a + b
```

PI = 3.14159

Save as "main.py":

CODE:

```
import mymodule
print(mymodule.greet("Krutik")) # Hello Krutik!
print(mymodule.add(5, 10))      # 15
print(mymodule.PI)              # 3.14159
```

```
# Or import specific
from mymodule import greet
print(greet("Rahul")) # Hello Rahul!
```

CHAPTER 21: OBJECT ORIENTED PROGRAMMING (OOP)

A) CLASS AND OBJECT:

CODE:

```
class Student:
    # Constructor
    def __init__(self, name, age):
        self.name = name
        self.age = age

    # Method
    def display(self):
        print(f"Name: {self.name}, Age: {self.age}")

    def greet(self):
        print(f"Hello, I'm {self.name}")

# Create Object
s1 = Student("Krutik", 25)
s2 = Student("Rahul", 22)

# Call methods
s1.display() # Name: Krutik, Age: 25
s1.greet()   # Hello, I'm Krutik
s2.display() # Name: Rahul, Age: 22
```

B) INHERITANCE:

CODE:

class Person:

def __init__(self, name, age):

self.name = name

self.age = age

def greet(self):

print(f"Hello, I'm {self.name}")

def details(self):

print(f"Name: {self.name}, Age: {self.age}")

class Student(Person):

def __init__(self, name, age, grade, course):

super().__init__(name, age) # Call parent constructor

self.grade = grade

self.course = course

Override method

def details(self):

**print(f"Name: {self.name}, Age: {self.age}, Grade: {self.grade},
Course: {self.course}")**

New method

def study(self):

print(f"{self.name} is studying {self.course}")

s = Student("Krutik", 25, "A+", "Python")

s.greet() # Hello, I'm Krutik (from parent)

**s.details() # Name: Krutik, Age: 25, Grade: A+, Course: Python
(overridden)**

s.study() # Krutik is studying Python

C) MULTIPLE INHERITANCE:

CODE:

```

class Teacher:
    def teach(self):
        print("Teaching...")

class Researcher:
    def research(self):
        print("Researching...")

class Professor(Teacher, Researcher):
    def teach(self):
        print("Professor is teaching...")

    def work(self):
        self.teach()
        self.research()

p = Professor()
p.teach()    # Professor is teaching...
p.research() # Researching...
p.work()    # Professor is teaching... Researching...

```

D) SPECIAL METHODS (Magic Methods):

CODE:

```

class Number:
    def __init__(self, value):
        self.value = value

    def __str__(self):
        return f"Number: {self.value}"

    def __repr__(self):
        return f"Number({self.value})"

    def __add__(self, other):
        return Number(self.value + other.value)

```

```
def __sub__(self, other):  
    return Number(self.value - other.value)
```

```
def __mul__(self, other):  
    return Number(self.value * other.value)
```

```
def __eq__(self, other):  
    return self.value == other.value
```

```
def __lt__(self, other):  
    return self.value < other.value
```

```
def __len__(self):  
    return len(str(self.value))
```

```
def __call__(self):  
    print(f"Called with value: {self.value}")
```

```
n1 = Number(10)
```

```
n2 = Number(20)
```

```
n3 = n1 + n2
```

```
n4 = n1 * n2
```

```
print(n3)      # Number: 30
```

```
print(n4)      # Number: 200
```

```
print(n1 == n2) # False
```

```
print(n1 < n2)  # True
```

```
print(len(n1))  # 2
```

```
n1()           # Called with value: 10
```

E) CLASS vs INSTANCE VARIABLES:

CODE:

```
class Student:
```

```
    college = "ABC University" # Class Variable (shared)
```

```
    total_students = 0
```



```
def __init__(self, name):
    self.name = name      # Instance Variable
    Student.total_students += 1

def display(self):
    print(f"Name: {self.name}, College: {Student.college}")
```

```
s1 = Student("Krutik")
s2 = Student("Rahul")
```

```
print(s1.college) # ABC University
print(s2.college) # ABC University
print(Student.total_students) # 2
```

```
# Change class variable
Student.college = "XYZ University"
print(s1.college) # XYZ University
print(s2.college) # XYZ University
```

F) PROPERTY DECORATORS:

CODE:

```
class Person:
    def __init__(self, name, age):
        self._name = name
        self._age = age

    @property
    def name(self):
        return self._name

    @name.setter
    def name(self, value):
        if not value:
            raise ValueError("Name cannot be empty")
        self._name = value
```

```

@property
def age(self):
    return self._age

```

```

@age.setter
def age(self, value):
    if value < 0:
        raise ValueError("Age cannot be negative")
    self._age = value

```

```

p = Person("Krutik", 25)
print(p.name) # Krutik
p.name = "Rahul"
print(p.name) # Rahul
# p.age = -5 # ValueError!

```

CHAPTER 22: POPULAR LIBRARIES (Introduction)

Library	Use
NumPy	Numerical computing, arrays
Pandas	Data analysis, DataFrames
Matplotlib	Data visualization, plots
Scikit-learn	Machine Learning
Django	Web Framework (Full Stack)
Flask	Micro Web Framework
Requests	HTTP requests (APIs)
BeautifulSoup	Web Scraping
Selenium	Browser Automation
PyTorch/TensorFlow	Deep Learning

OpenCV	Computer Vision
Tkinter	GUI Applications
SQLAlchemy	Database ORM
Celery	Task Queue
FastAPI	Modern Web Framework (APIs)

EXAMPLE - Requests:

CODE:

```
import requests
response = requests.get("https://api.github.com")
print(response.status_code) # 200
print(response.json())     # JSON data
```

EXAMPLE - NumPy:

CODE:

```
import numpy as np
arr = np.array([1,2,3,4,5])
print(arr.mean()) # 3.0
print(arr.sum())  # 15
```

EXAMPLE - Pandas:

CODE:

```
import pandas as pd
df = pd.DataFrame({"Name": ["Krutik", "Rahul"], "Age": [25, 22]})
print(df)
```

CHAPTER 23: QUICK COMMANDS CHEAT SHEET

STRING METHODS:

upper() lower() capitalize() title() strip() lstrip() rstrip()
split() join() replace() find() count() startswith() endswith()

isalpha() isdigit() isalnum() isspace() isupper() islower()

LIST METHODS:

**append() extend() insert() remove() pop() clear() index() count()
sort() reverse() copy()**

DICTIONARY METHODS:

keys() values() items() get() update() pop() popitem() clear() copy()

SET METHODS:

**add() remove() discard() pop() clear() union() intersection()
difference() symmetric_difference() issubset() issuperset()**

FILE METHODS:

read() readline() readlines() write() writelines() close()

BUILT-IN FUNCTIONS:

**len() type() print() input() int() float() str() list() tuple()
dict() set() bool() sum() min() max() abs() round() pow() sorted()
reversed() enumerate() zip() filter() map() range() open()**

MATH FUNCTIONS:

**abs() min() max() sum() round() pow() sqrt() sin() cos() tan()
log() exp() floor() ceil() factorial()**

PROJECT: STUDENT MANAGEMENT SYSTEM

CODE:

```
import os  
import json
```

```
class Student:  
    def __init__(self, name, age, grade):
```

```
self.name = name
self.age = age
self.grade = grade
```

```
def to_dict(self):
    return {"name": self.name, "age": self.age, "grade": self.grade}
```

```
def __str__(self):
    return f"Name: {self.name}, Age: {self.age}, Grade: {self.grade}"
```

```
class StudentManager:
```

```
    def __init__(self, filename="students.json"):
        self.filename = filename
        self.students = []
        self.load_data()
```

```
    def load_data(self):
        if os.path.exists(self.filename):
            with open(self.filename, "r") as f:
                data = json.load(f)
                self.students = [Student(**s) for s in data]
```

```
    def save_data(self):
        with open(self.filename, "w") as f:
            json.dump([s.to_dict() for s in self.students], f, indent=2)
```

```
    def add_student(self, name, age, grade):
        self.students.append(Student(name, age, grade))
        self.save_data()
        print(f"✓ Student {name} added!")
```

```
    def display_all(self):
        if not self.students:
            print("No students found!")
            return
        for i, s in enumerate(self.students, 1):
```

```
print(f"{i}. {s}")
```

```
def search_student(self, name):  
    found = False  
    for s in self.students:  
        if s.name.lower() == name.lower():  
            print(f"Found: {s}")  
            found = True  
    if not found:  
        print(f"Student '{name}' not found!")
```

```
def delete_student(self, name):  
    for i, s in enumerate(self.students):  
        if s.name.lower() == name.lower():  
            self.students.pop(i)  
            self.save_data()  
            print(f"✓ Student {name} deleted!")  
            return  
    print(f"Student '{name}' not found!")
```

Main Program

```
def main():  
    manager = StudentManager()  
  
    while True:  
        print("\n" + "="*40)  
        print("STUDENT MANAGEMENT SYSTEM")  
        print("="*40)  
        print("1. Add Student")  
        print("2. Display All Students")  
        print("3. Search Student")  
        print("4. Delete Student")  
        print("5. Exit")  
        print("="*40)  
  
        choice = input("Enter your choice (1-5): ")
```

```
if choice == "1":
    name = input("Enter name: ")
    age = int(input("Enter age: "))
    grade = input("Enter grade: ")
    manager.add_student(name, age, grade)

elif choice == "2":
    manager.display_all()

elif choice == "3":
    name = input("Enter name to search: ")
    manager.search_student(name)

elif choice == "4":
    name = input("Enter name to delete: ")
    manager.delete_student(name)

elif choice == "5":
    print("Goodbye! Happy Coding!")
    break

else:
    print("Invalid choice! Try again.")

if __name__ == "__main__":
    main()
```

CERTIFICATE OF COMPLETION

This certifies that the reader has completed

PYTHON COMPLETE COURSE

By
KRUTIK  CYBER EXPERT

Topics Covered:

- ✓ Python Basics
- ✓ Variables & Data Types
- ✓ Strings & String Methods
- ✓ Lists, Tuples, Sets, Dictionaries
- ✓ If-Else & Loops
- ✓ Functions & Lambda Functions
- ✓ File Handling
- ✓ Exception Handling
- ✓ Modules & Packages
- ✓ Object Oriented Programming (OOP)
- ✓ Popular Libraries (NumPy, Pandas, etc.)
- ✓ Project: Student Management System

"Code Secure. Stay Safe."

NOTE: This PDF is completely FREE. If you paid for it, you got scammed.

Share it with everyone who wants to learn Python! 
